CS2106 Finals Cheatsheet

Anselm Long AY25/26

Adapted from weiseng18 and zhuhanming

OS Structure

OS runs in **kernel mode** (full hardware access); user programs run in **user mode** (controlled access). Switching from user→kernel mode (t1) is cheaper than a full process context switch (t2): **t1 < t2**.

Kernel

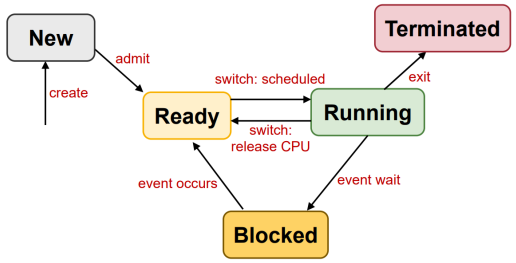
Monolithic - One large program; all services integral. Traditional (Unix, Windows NT/XP).
✓ Well understood; good performance. × Highly coupled; complex internal structure.
Microkernel - Very small kernel; only IPC, address space management, thread management. Higher-level services run as *server processes* in user space; communicate via IPC.
✓ Robust; extensible; better isolation/protection. × Lower performance (more IPC overhead).
Layered: components in a hierarchy; upper layers use lower layers.
Client-Server: variant of microkernel; client requests service from server processes (can be on different machines).

Processes

A **process** is an abstraction for a running program, identified by a **PID**.
Memory regions: Text (instrs), Data (globals), Heap (dynamic alloc), Stack (fn calls).
Transient data: valid for limited duration (fn params, locals).
Persistent data: valid for program duration or until explicitly freed (globals, dynamic alloc).

Process State (5-Stage)

Generic 5-State Process Model



New → Ready → Running ↔ Blocked; Running → Terminated.
Transitions: (1) Create: NIL→New (2) Admit: New→Ready (3) Switch: Ready→Running (4) Switch: Running→Ready (5) Event Wait: Running→Blocked (6) Event Occurs: Blocked→Ready
For Unix: Stopped: STOP signal → process suspended; CONT signal → resumes to Ready.
PCB stores entire execution context; PCBs stored in **Process Table** by kernel.

Syscall Mechanism

Function wrapper = same name/params as syscall. **Function adapter** = user-friendlier.
(1) Program invokes library call → (2) library places syscall# in register → (3) executes TRAP (kernel mode) → (4) dispatcher finds handler → (5) handler executes → (6) return to user mode → (7) library returns to program.
Exceptions: synchronous → exception handler.
Interrupts: asynchronous → program suspended, interrupt handler runs.
Handler routine: save state → perform routine → restore state → return from interrupt.

UNIX Process Calls

`init`: root of all processes, PID 1, created at boot.
`fork()`: creates child with copy of parent image. Data NOT shared. Returns 0 to child, child PID to parent.
Linux `clone()` supersedes `fork()`: allows partial duplication (shared memory regions, fds, etc.).
fork() steps: (1) allocate new PCB+PID (2) copy memory regions (3) acquire shared resources (files, CWD) (4) init hardware context from parent (5) add child to scheduler queue.
`exec()`: replaces current process image. PID and other info remain; only code changes.
`exit(status)`: terminates process, returns status to parent. PID/CPU time/PCB remain until parent calls `wait()`.
`wait()`: blocks until child finishes; cleans up child PCB. Returns −1 if no child.
Zombie: child finished but parent hasn't called `wait()`.
Orphan: parent terminated before child. `init` adopts child. *Distinct from zombie* — orphan's parent is simply gone.

Process Scheduling

Pseudo-parallelism (1 CPU) or **physical parallelism** (multi-CPU).
Common criteria: Fairness (fair CPU share, no starvation); Balance (all parts utilised).
Non-preemptive: runs until blocks/yields. **Preemptive:** fixed time quantum, can be interrupted.
Starvation: when a process is ready to run but is indefinitely denied CPU time because the scheduling algorithm keeps choosing other processes over it

Process Behavior

A typical process alternates between:
CPU-activity: computation (e.g. number crunching). **Compute-bound** processes spend most time here.
I/O-activity: waiting for I/O devices. **I/O-bound** processes spend most time here.

Processing Environments

(1) Batch: no user interaction; no need to be responsive; non-preemptive predominant.
(2) Interactive: active user; must be responsive and consistent in response time; preemptive.
(3) Real-time: has deadlines to meet; usually periodic processes (e.g. aircraft control, media players).

Batch Processing

No user interaction; non-preemptive predominant.
Criteria: Turnaround = finish−arrival; Throughput = tasks/unit time; CPU util = % busy.

FCFS

FIFO by arrival time. ✓ No starvation. × Convoy effect: long task inflates turnaround for all behind it.

SJF (Shortest Job First)

Select smallest *total* CPU time. ✓ Min avg wait time. × Possible starvation. × Hard to predict.
Estimate: $P_{n+1} = \alpha \cdot A_n + (1-\alpha) \cdot P_n$

SRT (Shortest Remaining Time)

Preemptive SJF. New short job can preempt long-running one. If all jobs arrive at the beginning, same schedule as SJF.

Interactive Scheduling

Preemptive; goal = good response times.
Criteria: Response time = first response − request time; Predictability = low variance.
Timer Interrupt: fires periodically, cannot be intercepted, triggers scheduler. ITI: 1–10ms. Time quantum: 5–100ms (multiple of ITI).

Round Robin (RR)

FIFO; fixed quantum per process. Blocked tasks move to wait queue; rejoin end when ready.
Max wait before CPU = $q(n-1)$ for n tasks, quantum q.
Large q → better util, longer wait. Small q → more overhead, shorter wait.

Priority Scheduling

Highest priority runs. Can be preemptive or not.
× Starvation. Fixes: (1) decrease priority of running process each quantum; (2) stop after quantum, exclude from next round.
Priority inversion: lower-priority task holds resource needed by higher-priority one.

MLFQ

Multiple levels with their own priority but processes move between queues based on behaviour.
Rules: (1) Priority(A)>Priority(B) ⇒ A runs. (2) Equal priority ⇒ RR. (3) New job → max priority. (4) Uses full quantum ⇒ priority reduced. (5) Gives up/blocks early ⇒ priority maintained.
✓ Adaptive. ✓ Minimises response time (I/O-bound) and turnaround (CPU-bound).
× Abuse: yield just before quantum expires to stay at high priority. Fix: periodic priority boost.

Lottery Scheduling

Give processes lottery tickets; pick random winner for CPU.
✓ Responsive (new processes join immediately). ✓ Control via ticket distribution. ✓ Simple.

Inter Process Communication (IPC)

Shared Memory

P1 creates region; P2 attaches. Both read/write directly.
✓ Efficient (only setup needs OS). ✓ Easy to use. × Needs explicit sync. × Hard to implement.
`shmget()` create (returns ID); `shmat()` attach (returns ptr); `shmdt()` detach; `shmctl()` destroy.

Message Passing

Stored in kernel; send/receive are syscalls.
Direct: sender/receiver name each other (1 link/pair). **Indirect**: via named **port** (shareable).
Blocking (sync): send blocks until received; recv blocks until message arrives.
Non-blocking (async): send returns immediately; recv returns message or "none".
✓ Portable. ✓ Easy sync with blocking. × Needs OS each time. × Limited size/format.

Pipe (UNIX)

FIFO circular bounded byte buffer. Writers block when full; readers block when empty. Close unused ends.
`pipe(fd[2])`: `fd[0]`=read end, `fd[1]`=write end.
`close(fd)`: closes fd. 0=stdin, 1=stdout, 2=stderr.
`dup(fd)`: duplicates to lowest unused fd (alias). `dup2(fd,newfd)`: specifies target fd.

Signal (UNIX)

Async notification to process/thread. Default or user-supplied handler.
`signal(signum, handler)`: replace default. Some signals unreplaceable (e.g. `SIGKILL`).

Threads

Motivation: `fork()` + context switch expensive; independent address spaces make comms harder.

Thread Model

Shared: memory context (Text, Data, Heap); OS context (PID, open files).
Own: thread ID; registers (GP + special); stack.
Thread switch: hardware context only (regs, FP, SP) — much cheaper than process switch.
Benefits: Economy, resource sharing, responsiveness, scalability (multi-CPU).
Issues: parallel syscalls need correctness; `fork()` creates 1 thread in child (Linux); single thread `exit()` kills all; single thread `exec()` replaces image.

User Threads

Library-level; kernel unaware. ✓ Any OS; just library calls; configurable policy. × One thread blocks ⇒ all block. Cannot use multi-CPU.

Kernel Threads

OS-level; syscall-managed. ✓ Multi-CPU; thread-level scheduling. × Slow (syscall overhead); less flexible.

Hybrid Model

OS schedules kernel threads; user threads bind to kernel threads.
Simultaneous Multi-Threaading (SMT): Hardware-level multiple register sets per core (e.g. Intel Hyperthreading).

POSIX pthreads

`pthread_create(&tid, attr, fn, arg)`: `fn` signature = `void* fn(void* arg)`. This will call fn(arg) in the new thread.
`pthread_exit(retval)`: terminate thread.
`pthread_join(tid, &status)`: wait for thread, get exit value.
Mutex: `pthread_mutex_lock()`, `pthread_mutex_unlock()` — binary semaphore equivalent.
Condition var: `pthread_cond_wait()`, `pthread_cond_signal()`, `pthread_cond_broadcast()`.

Synchronisation

Race condition: concurrent processes execute in an interleaving fashion and access a shared resource.

Critical Section

Region where only 1 process may execute at a time. Place with a race condition.

Properties:

1. **Mutual Exclusion** — only 1 process in CS.
2. **Progress** — if no process in CS, one must be admitted.
3. **Bounded Wait** — finite upper bound to wait before P enters.
4. **Independence** — non-CS processes don't block others.

Incorrect Sync Symptoms

Deadlock: all blocked; no progress.

1. **Mutual Exclusion** — resource is held in a non-shareable mode; only one process can use it at a time.
2. **Hold-and-wait** — a process holds at least one resource while waiting to acquire additional resources held by others.
3. **No preemption** — resources cannot be forcibly taken away; a process must release them voluntarily.
4. **Circular wait** — a cycle exists in the resource allocation graph

Livelock: processes keep changing state to avoid deadlock but make no actual progress. *Processes are NOT blocked* — they are actively running but achieving nothing.

Starvation: perpetually denied resources.

CS Implementation Tiers

Three tiers of critical section implementation: **(1) Low-level (assembly):** TestAndSet, Peterson's — busy waiting, hardware-dependent.

(2) High-level language: use normal programming constructs (e.g. disable interrupts). Still limited; only useful in specific contexts.

(3) High-level abstraction: semaphores, condition variables — specify behaviour not implementation; most general and preferred.

Low-Level Implementations

Test-And-Set

Atomic: loads memory value to register AND sets memory to 1. Satisfies all 4 CS properties. × Busy waiting. Variants: Compare-And-Exchange, Atomic Swap, LL/SC.

Peterson's Algorithm

Software; 2 processes; assumes atomic write to `Turn`.

```
// P0: Want[0]=1; Turn=1;
//     while(Want[1] && Turn==1); /* CS */ Want[0]=0;
// P1: Want[1]=1; Turn=0;
//     while(Want[0] && Turn==0); /* CS */ Want[1]=0;
```

× Busy waiting. × Only 2 processes.

Semaphores & Mutexes

Protected non-negative integer S.

General/counting: $S \geq 0$. **Binary/mutex:** $S \in \{0, 1\}$.

`wait(S)`: if $S \leq 0$, block *(before decrement)*; decrement S. Note: S can go negative; $|S|$ = number of blocked processes. `signal(S)`: wake one sleeping process (if any); increment S. Atomic; never blocks.

Invariant: $S_{cur} = S_{init} + \#\text{signal}(S) - \#\text{wait}(S)$

Deadlock via incorrect use: P0 does `wait(P)` then `wait(Q)`; P1 does `wait(Q)` then `wait(P)` ⇒ deadlock if P0 holds P and P1 holds Q.

Condition Variable

Process waits for specific event; broadcast on occurrence wakes all waiting tasks.

Classical Problems

Producer-Consumer: bounded buffer size K. Semaphore for empty slots, full slots, mutex for buffer. Producers insert only when not full; consumers remove only when not empty.

Producer Consumer: Blocking Version

```
while (TRUE) {
        Produce Item;

        wait( notFull );
        wait( mutex );
        buffer[in] = item;
        in = (in+1) % K;
        count++;
        signal( mutex );
        signal( notEmpty );
}
```

Producer Process

```
while (TRUE) {

        wait( notEmpty );
        wait( mutex );
        item = buffer[out];
        out = (out+1) % K;
        count--;
        signal( mutex );
        signal( notFull );

        Consume Item;
}
```

Consumer Process

- Initial Values:
  - `count = in = out = 0`
  - `mutex = S(1), notFull = S(K), notEmpty = S(0)`

Readers-Writers: readers concurrent; writers exclusive. Prevent writer starvation.

Readers Writers: Simple Version

```
while (TRUE) {

    wait( roomEmpty );

    Modifies data

    signal( roomEmpty );
}
```

Writer Process

```
while (TRUE) {

    wait( mutex );
    nReader++;
    if (nReader == 1)
        wait( roomEmpty );
    signal( mutex );

    Reads data

    wait( mutex );
    nReader--;
    if (nReader == 0)
        signal( roomEmpty );
    signal( mutex );

}
```

Reader Process

- Initial Values:
  - `roomEmpty = S(1)`
  - `mutex = S(1)`
  - `nReader = 0`

Dining Philosophers: 5 philosophers, 5 chopsticks. Each needs both adjacent. Naïve (all grab left) → deadlock. Fix (put down left if right unavailable) → livelock. Correct: asymmetric ordering / waiter (mutex) / limit to $N-1$ concurrent eaters.

```
// Init: state[N]=THINKING, Semaphore mutex=1, Semaphore s[N];
// LEFT=(i+N-1)%N  RIGHT=(i+1)%N, N=5
```

```
void philosopher(int i) {
  while (TRUE) {
    Think();
    takeChpStcks(i);
    Eat();
    putChpStcks(i);
  }
}
```

```
void safeToEat(int i) {
  if (state[i]==HUNGRY &&
      state[LEFT]!=EATING &&
      state[RIGHT]!=EATING){
    state[i] = EATING;
    signal(s[i]);
  }
}
```

```
void takeChpStcks(int i) {
  wait(mutex);
  state[i] = HUNGRY;
  safeToEat(i);
  signal(mutex);
  wait(s[i]);
}
```

```
void putChpStcks(int i) {
  wait(mutex);
  state[i] = THINKING;
  safeToEat(LEFT);
  safeToEat(RIGHT);
  signal(mutex);
}
```

Race Condition: Counting Interleavings

Each high-level statement = multiple atomic ops (Load, Add, Store, etc.).

Method: enumerate all possible orderings of atomic ops across processes; count distinct outcomes.

Exam pattern: Count number of orderings → eg. P: `x=x+1; x=x/2` (4 ops: L,A,L,S or similar) + Q: `x=x-3` (L,S). For n ops in P and m ops in Q: total interleavings = $\binom{n+m}{m}$; distinct *values* are usually fewer.

Barrier: synchronization for n processes — all block until n-th arrives, then all unblock simultaneously. Implemented with a mutex + counting semaphore initialized to 0.

Memory Management

Memory Abstraction

No abstraction: prog addr = phys addr. × No protection; conflicts between processes.

Address relocation: recalculate all addresses at load time. × Slow loading; can't distinguish addr from int.

Base + Limit registers: all refs offset by Base; checked against Limit. × Addition + comparison per access.

Logical address: process's independent view; OS maps to physical.

Contiguous Allocation

Internal Fragmentation - wasted space inside an allocated region

External Fragmentation - wasted space outside all allocations

Fixed-Size Partitions

Fixed equal partitions; 1 process/partition. ✓ Easy; fast. ✓ No external frag. × Must fit largest process. × Internal frag.

Variable-Size (Dynamic) Partitions

Exact-fit partitions. ✓ No internal frag. × External frag. × More bookkeeping.

Reduce ext. frag: Merging (merge adjacent free); Compaction (move partitions, expensive).

Dynamic Allocation Algorithms

All: no internal frag; have external frag.

First-Fit: first hole $\geq N$. **Best-Fit:** smallest hole $\geq N$. **Worst-Fit:** largest hole.

Next-Fit: like first-fit but resume from last allocation. ✓ Uniform hole sizes; faster. × Extra pointer.

Partition info: linked list (status, start addr, length, next ptr) or bitmap.

Buddy System

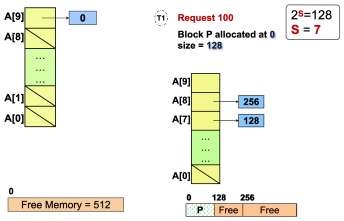
Array $A[0 \ldots K]$ where $A[j]$ = linked list of free blocks of size 2^j. 2^K = largest block.

Allocate N: find smallest S s.t. $2^S \geq N$; if $A[S]$ has block, use it; else find smallest $R > S$ with a block and split down to 2^S, adding halves to $A[S \ldots R-1]$.

Deallocate block B **(size** 2^S**):** find buddy C (differs only in bit S of start addr); if C free, merge → repeat; else insert B into $A[S]$.

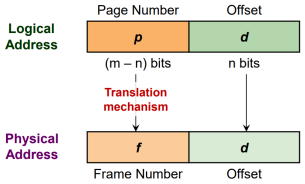
Buddy check: two blocks of size 2^S are buddies ⇔ only bit S of start addresses differs.

× Internal frag up to ∼50%. × Small external frag possible.



Paging

Physical memory → fixed-size **frames**; logical memory → equal-size **pages**. Logical contiguous; physical disjoint.



Address translation (page/frame size 2^n, address m bits): top $(m-n)$ bits = page# p; bottom n bits = offset o. Lookup p in page table → frame# f. Physical addr = $f \| o$.

✓ No external frag. × Internal frag (last page may be partial). ✓ Clean logical/physical separation.

Page Table: page# → frame#; stored in PCB. × 2 RAM accesses/reference: (1) read PTE; (2) access data.

TLB (Translation Look-Aside Buffer)

On-chip cache of PTEs. Tens of entries; ≤ 1 clock cycle. Part of **hardware context** → flushed on context switch.

Hit: frame retrieved directly. **Miss:** access full page table in RAM; update TLB.

Let p = hit probability, T_t = TLB time, T_m = RAM time:

$$\text{Avg} = p(T_t + T_m) + (1-p)(T_t + 2T_m) = T_t + T_m(2-p)$$

Page Table Protection

Access-right bits (rwx): checked in hardware on every access.

Valid bit: out-of-range access caught by OS.

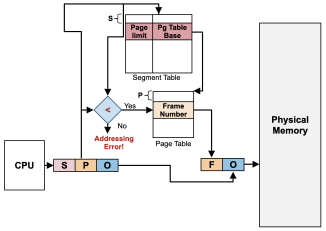
Page Sharing

Multiple pages → same frame. **Shared bit** in PTE.

COW (Copy-On-Write): after `fork()`, parent and child share frames, page table entries point to the same entry (page table copied, frames not). On write to shared page: duplicate frame, clear shared bit.

Segmentation
Motivation: memory regions have different sizes, lifetimes, permissions; can grow/shrink.
Each segment: (*seg id*, *base*, *limit*). Ref: (*seg id*, *offset*). Physical = base + offset if offset < limit; else **segfault**.
Segment table: array of [Base, Limit] indexed by seg ID; stored in registers. × External frag.
Key exam trap: process address space does NOT need to be contiguous in pure segmentation; each *segment* must be contiguous. Segments from different processes can be adjacent in physical memory.

Segmentation with Paging
Each segment has its own page table. Segment table → page table base. Grow segment by adding pages. Worst case internal fragmentation = $n \times$ (page size $-$ 1), where n is number of segments.



Virtual Memory
Some pages in RAM; others on disk. Logical addr space not bounded by physical RAM.
Extended paging: add **"is memory resident?"** bit to PTE. CPU access to non-resident page ⇒ **page fault**.

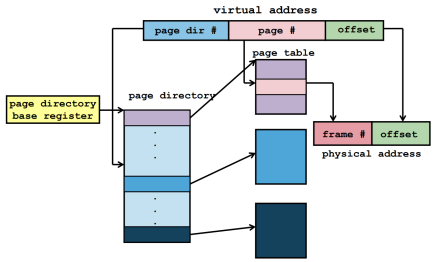
Virtual Address Access Algorithm
Done in hardware:
(1) Decompose VA into `<Page#, Offset>`.
(2) Search TLB for `Page#`. If TLB miss → TRAP (TLB Fault Handler), retry.
(3) Is `Page#` memory resident? If not → TRAP (Page Fault Handler), retry.
(4) Use `<Frame#><Offset>` to access physical memory.
TLB Fault Handler (SW-managed TLB, done in OS):
(1) Find `Page#` in full page table (PCB). (2) Valid bit = 0 → segfault. (3) Load PTE into TLB. (4) Return from TRAP.
TLB Fault Handler (HW-managed TLB):
(1) Base of page table in dedicated register. (2) Processor retrieves PTE from memory directly.
Page Fault Handler (done in OS):
(1) Global or local replacement? (2) Write out victim if dirty. (3) Locate page in secondary storage. (4) Load page into frame. (5) Update PTE. (6) Update TLB. (7) Return from TRAP. *Note: page fault counts as I/O.*

Demand Paging
Start with no resident pages; load only on fault. ✓ Fast startup; small footprint. × Sluggish initially; can cause thrashing.
Thrashing: repeated page faults from insufficient frames. Avoided by temporal/spatial **locality**.
Temporal Locality - memory address which is used is likely to be used again
Spatial Locality - memory addresses close to a used address is likely to be used

Page Table Structures
Assume 2^P pages in logical address space.

Direct (Single-Level)
2^P entries in one contiguous table. × Must be contiguous in OS memory. Wasteful if sparse.
32-bit, 4KB pages, 2B entries: $2^{20} \times 2$B = 2MB per process.

2-Level Paging



"Page the page table." Page directory: 2^M entries → 2^M page tables. Each page table: 2^{P-M} entries.
✓ Not contiguous in OS memory. ✓ Empty regions = null directory entry. × 2 RAM accesses to get frame#.
MMU cache caches page directory entries; TLB caches PTEs.
Overhead example (32-bit, 4KB pages, 2B entries): offset = 12 bits; 20 bits remain. Each page table = 4KB → 2^{11} entries. Directory = 2^9 entries → 1KB overhead. Total = 1KB + (allocated page tables × 4KB).

Inverted Page Table
1 entry per physical frame; stores `<PID, page#>`. 1 table for all processes.
✓ Good for frame-level management. × Slow translation (search whole table). O(1) with hashing, O(n) worst case Used as auxiliary structure. Size is number of physical frames not virtual pages.

Page Replacement Algorithms
Memory access time: Let T_{pf} be time for page fault. p is probability of page fault. $T_{access} = (1-p) \cdot T_{mem} + p \cdot T_{pf}$. Minimise p since $T_{pf} \gg T_{mem}$.
Memory reference string: sequence of page numbers (offsets irrelevant).
Clean page: not modified → no write-back. **Dirty page:** modified → must write back.

OPT (Optimal)
Replace page not needed for longest time. ✓ Min page faults; benchmark. × Not feasible (needs future knowledge).

FIFO
Evict oldest page (queue by load order). ✓ Simple; no HW support. × Bad perf.
× **Bélady's Anomaly**: more frames can ↑ page faults (e.g. `1 2 3 4 1 2 5 1 2 3 4 5` with 3 vs 4 frames).

LRU
Replace least recently used page. ✓ Exploits temporal locality. ✓ No Bélady's. × Needs substantial HW support.
Impl 1 (time counter): store `lastUsedTime` in PTE; update every access. Query = find min counter. × $O(n)$; overflow risk.
Impl 2 (stack): on reference, pop page from stack and push to top; replace bottom. × Not true stack (pop from middle); hard in HW.

Second-Chance (CLOCK)
Modified FIFO + **reference bit** per page (1=accessed, 0=not). Circular queue; pointer to oldest (victim).
On **load**: ref bit = 0. On **access**: ref bit = 1.
On **replacement**: if victim ref bit = 0 → replace; if ref bit = 1 → clear to 0, advance pointer (second chance), repeat. Degenerates to FIFO when all bits = 1.

Frame Allocation
N frames, M processes.
Equal: N/M per process. **Proportional:** $(size_p/size_{total}) \times N$.
Local replacement: victim from faulting process only. Thrashing limited to 1 process; consistent perf; × insufficient frames hinder progress.
Global replacement: victim from any process. ✓ Self-adjusting. × Inconsistent perf; cascading thrashing.

Working Set Model
$W(t, \Delta)$ = set of distinct pages referenced in interval ending at t (window = Δ).
Allocate enough frames for $W(t, \Delta)$. Δ too small → miss pages in locality. Δ too large → pages from multiple localities.
Example (Δ=5): $W(t_1, \Delta) = \{1, 2, 5, 6, 7\}$ (5 frames); $W(t_2, \Delta) = \{3, 4\}$ (2 frames).

All PTE Bits
(1) Access Right Bits (rwx) (2) Valid Bit (3) Is-Shared Bit (4) Is-Memory-Resident Bit (5) Dirty Bit (6) Reference Bit (CLOCK)

Exam Calculation Patterns
PTE & Page Table Size
PTE bits = frame# bits + status bits (rwx, valid, dirty, resident...).
Round up to nearest byte.
Page table size = $2^p \times$ PTE size where p = # page bits.
Internal frag = page size $-$ (process size mod page size); if remainder = 0, frag = 0.
e.g. 72766 bytes, 2KiB page: 72766 mod 2048 = 1086; frag = 2048 $-$ 1086 = 962 bytes.

N-level Paging TLB Access Time
TLB miss cost = $T_{tlb} + N \times T_{mem}$ (index levels) + T_{mem} (data).
Avg = $h(T_{tlb} + T_{mem}) + (1-h)(T_{tlb} + (N+1)T_{mem})$
e.g. 4-level, h=80%, T_{tlb}=20ns, T_{mem}=100ns: miss=20+500=520; avg=0.8(120)+0.2(520)=**200ns**.

Combined TLB + Page Fault Time
TLB hit ⇒ page guaranteed in memory (no fault). Let p_f = page fault rate (among TLB misses).
$T = h(T_{tlb} + T_{mem}) + (1-h)[T_{tlb} + (1-p_f)T_{mem} + p_f \cdot T_{pf}]$
Worst case = TLB miss + page fault with dirty page:
$T_{tlb} + T_{mem} + 2T_{disk} + T_{mem}$ (page-table lookup + writeback + reload + re-access).

Buddy System Address Check
Two blocks of size 2^S are buddies ⇔ only bit S of their start addresses differs.
e.g. 1KiB buddies (S=10): `0xF400` and `0xF800` differ at bit 11 — NOT buddies. `0xF400` and `0xFC00` differ only at bit 10 — buddies. ✓
To find starting address of a buddy block: mask off the lower S bits of any address within it.

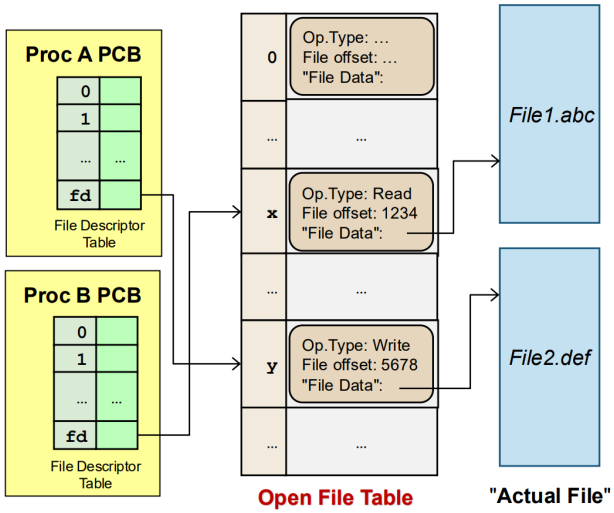
File Systems
General criteria: Self-Contained (plug-and-play); Persistent (beyond processes); Efficient (min overhead).

File
Logical unit of info. **Metadata:** name, id, type, size, protection, time/date/owner.
Windows → file extension; UNIX → **magic number** at file start.
Types: Regular (ASCII: printable; Binary: predefined structure); Directories; Special (char/block devices).
Data structures: Array of bytes; Fixed-length records (direct jump); Variable-length records (flexible, hard to locate).
Records: C file = char; SQL = row; video = compressed frame (variable length).
Access methods: Sequential (in-order); Random (`Read(offset)` or `Seek(offset)+Read`); Direct (random by record).
Ops: Create, Open, Read, Write, Reposition (Seek), Truncate; Rename, Change/Read attributes.

File Protection
Access types: Read, Write, Execute, Append, Delete, List (read metadata).
Unix permission bits: Owner/Group/Universe, RWX each. ACL: minimal (= perm bits) or extended (named users/groups).
Symlinks always show 777; actual access checked against **target file**.
Directory permissions:
rwx means different things on a directory vs a file: r=list contents; w=modify entries (create/rename/delete); x=use as CWD (`cd`, traverse).

Action	r-x	-wx	–x
`ls -l DIR`	ok	no	no
`cd DIR`	ok	ok	ok
`ls -l` (after cd)	ok	no	no
`cat curfile`	ok	ok	ok
`touch curfile`	ok	ok	ok
`touch newfile`	no	ok	no

File Operations & Open-File Table
OS provides file ops as syscalls for protection, concurrency, efficiency.
Info per open file: file pointer (current pos); disk location; open count.



Two-table approach:
(1) **System-wide open-file table:** 1 entry per unique open (op type, offset, file data ptr).
(2) **Per-process fd table:** 1 entry per fd; points to system-wide entry.
Case 1 (independent): 2 separate `open()` calls → 2 system-wide entries → independent offsets.
Case 2 (shared): same system-wide entry → shared offset. Via `fork()` (child inherits fd table) or `dup()/dup2()` (alias within same process).

UNIX FS Calls

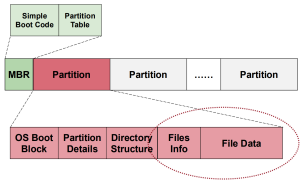
`open(path,flags)`: returns fd.
`read(fd,buf,n)`: reads $\leq n$ bytes from offset; increments offset.
`write(fd,buf,n)`: writes $\leq n$ bytes at offset; increments offset. Appends past EOF.
`lseek(fd,offset,whence)`: `SEEK_SET`=absolute; `SEEK_CUR`=relative from pos; `SEEK_END`=relative from end.
`close(fd)`: closes fd; all fds auto-closed on termination.

Directory

Purpose: logical grouping + track files. Structures: single-level, tree, DAG, general graph.
Traversal: given full path, recursively check each component (correct type); stop if not found.
Hard link: multiple dir entries → same inode. Files only; creates DAG. ✓ Low overhead. × Deletion needs ref count. (`ln`)
Soft (symbolic) link: file containing target pathname. Files+dirs; creates cycles. ✓ Simple deletion. × Extra inode/disk space; dangling if target deleted. (`ln -s`)
Dir implementation: Linear list (1 entry/file; linear search; can cache); Hash table (hash filename, chained collision; fast; fixed size; needs good hash).
File info storage: (1) everything in dir entry; (2) filename + ptr to separate DS.

FS Implementation

OS views disk as 1-D array of **logical blocks** (512B–4KB). Hardware maps to sectors.
Layout: MBR (sector 0; boot code + partition table) | Partition 1 | Partition 2 | . . .
Each partition: OS boot block, partition details, dir structure, file info, file data.



Note: all allocation methods can have internal frag if last block of a file is not full.

Block Allocation Methods

Contiguous

Consecutive disk blocks. File info = (start, length). ✓ Simple; fast access. × Ext. frag. × File size must be known.

Linked List

Each block stores data + pointer to next. File info = first + last block. ✓ No ext. frag. × Slow random access $O(n)$. × Pointer wastes space. × One bad ptr → data loss.

FAT (Linked List v2)

All pointers in File Allocation Table (RAM). Block index = FAT index. Block = cluster. ✓ Faster random access (traversal in RAM). × FAT tracks all blocks; huge for large disks.

Indexed

One index block per file = array of all block addresses. ✓ Less memory overhead (only open file index blocks). ✓ Fast direct access. × Max file size limited by index block capacity. × Index block overhead.

Indexed + Linked List

Chain of index blocks; each contains ptr to next index block. Extends indexed to larger files.

Multilevel Indexing

Like multilevel paging for blocks. Direct + single/double/triple indirect pointers.
Unix I-node: 12 direct + 1 single + 1 double + 1 triple indirect.
(4B addresses, 1KB blocks ⇒ 256 addresses/block)
$12 \times 1\text{KB} + 256 \times 1\text{KB} + 256^2 \times 1\text{KB} + 256^3 \times 1\text{KB} \approx 16\text{GB}$ max file size.

Free Space Management

Bitmap

1 bit/block (1=free, 0=occupied). ✓ Powerful bit ops. × Must keep in memory.

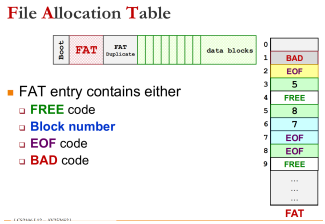
Linked List of Free Blocks

Each disk block holds free block numbers + ptr to next free-list block. ✓ Easy; only first ptr in memory. × High overhead. Mitigation: store list in free blocks. **Runs alternative:** store (first_index, length) pairs for contiguous runs.

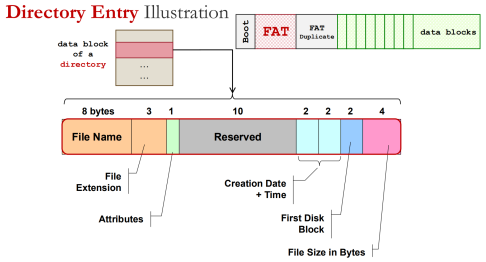
FS Case Studies

FAT (MS-DOS)

Layout: Boot | FAT | FAT Duplicate (optional) | Root Dir | Data Blocks
FAT entry values: FREE (unused); block# of next block; EOF; BAD (disk error).



Directory: special file type. Root dir at fixed location; others in data blocks.
Directory entry (32 bytes fixed):



Field	Size	Notes
File Name	8B	0xE5=deleted; 0x00=end; 0x2E=parent
Extension	3B	
Attributes	1B	RO, Dir/File, Hidden. . .
Reserved	10B	
Date+Time	4B	1980–2107; ±2s accuracy
First Block	2B	12/16/32 bits for FAT12/16/32
File Size	4B	

Max partition size = cluster size × max clusters.
FAT32: 2^{28} clusters × cluster size. With 1KiB clusters: $2^{28} \times 1\text{KiB} = 256\text{GiB}$.
FAT16: 2^{16} clusters. With 4KiB clusters: $2^{16} \times 4\text{KiB} = 256\text{MiB}$.
FAT tracing: (1) Get dir table. (2) Find entry → first block. (3) Follow FAT chain to EOF. (4) Access data blocks (repeat for subdirs).
File deletion: set filename[0]=0xE5; set FAT chain to FREE. Data intact → recoverable.
Free space management: walk entire FAT.

FAT Variants

Disk cluster: group of contiguous blocks as smallest alloc unit.
Bigger FAT ⇒ more bits/entry ⇒ more clusters addressable.

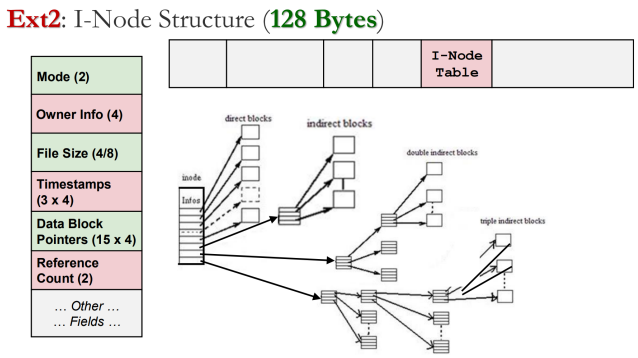
	FAT12	FAT16	FAT32
Max clusters	2^{12}	2^{16}	2^{28}
Max (4KiB)	16MiB	256MiB	1TiB

Larger cluster → larger partition; × larger internal frag.
FAT32: only 28/32 bits used in cluster index; 32-bit sector count limits size.
VFAT: filenames up to 255 chars. Uses multiple 32-byte dir entries per long-name file. Invalid file attribute byte → non-VFAT apps ignore entries. First byte = sequence number to stitch entries. (13 Unicode chars/entry across multiple fields.)

Linux Ext2

Layout: MBR | Boot | Block Group 0 | Block Group 1 | . . .
Each **block group** contains:
(a) **Superblock**: entire FS info (total inodes, blocks, etc.). Duplicated per group.
(b) **Group Descriptors**: per-group free blocks, free inodes, bitmap locations. Duplicated.
(c) **Block Bitmap**: 1=occupied, 0=free; 1 bit/block.
(d) **I-Node Bitmap**: 1=occupied, 0=free; 1 entry/inode slot.
(e) **I-Node Table**: array of inodes.
(f) **Data Blocks**.

I-Node Structure (128 bytes)



Field	Size	Description
Mode	2B	type + permissions
Owner Info	4B	UID(2B) + GID(2B)
File Size	4/8B	bytes (8B for larger files)
Timestamps	12B	Create/Modify/Delete (3×4B)
Data Ptrs	60B	12 direct+1 SI+1 DI+1 TI (15×4B)
Ref Count	2B	# dir entries pointing here

✓ Fast small file access (12 direct). ✓ Flexible large file handling (indirect).

Inode Disk Access Count Calculation

Given inode in memory, reading B data blocks:
Direct (≤ 12): no extra reads. 1 single-indirect block spans k blocks (k=block_size/ptr_size).
Total reads = (index blocks traversed) + B data block reads.
e.g. 512B blocks, 16-bit ptrs (k=256): read 4096 blocks from 4-direct+2-SI+1-DI inode = 4 + 512 (2 SI index reads + 512 data) + $1+\lceil 3580/256 \rceil = 15$ (DI reads) + 3580 data = 4113 total.

Ext2 Directory Structure

Dir is a file; data blocks = linked list of variable-length entries:
(1) I-Node# (0=unused) (2) Entry size (3) Name length (4) Entry type (5) Name (up to 255 chars).

Ext2 Path Traversal

(1) Get root inode (#2). (2) For each subdir in path: scan dir data blocks → get inode# → read inode → descend. (3) For target file: scan → get inode# → read inode (metadata + data ptrs).

Higher-Level Ops

Open: traverse path; load inode into system-wide table entry E; create fd in process table → E; return fd.
Delete: remove dir entry (relink prev→next; blank if first); mark inode free in bitmap; mark blocks free in bitmap.
Hard Link: new dir entry → same inode#; increment ref count. Delete ⇒ decrement; actual deletion at count=0.
Symbolic Link: create new file Y (new inode + dir entry); store path of X as Y's content. × Invalidated if X renamed/moved/deleted.